Uinsure Policies API
User guide

Two journeys through the API: a developer joining the project, who clones, runs, tests and learns where a rule lives, and a servicer who drives the API directly with curl and Postman. At every step you see what you decide, what the repository or the API gives you, and the real output it produces.

Every command in this guide was run on the feat-001/N1 branch on 2026-10-25, and every output, count and test number is copied from that run (`receipts.md`, next to this guide's source).

The big ideas

1. **A policy has a life.** It is sold, is pending or active, can be renewed in its last 30 days, and ends cancelled or expired. Its status is worked out from today and never stored.
2. **Four projects, one direction.** The domain holds every rule and needs only .NET. Services orchestrate, Persistence stores, the Api speaks HTTP. Architecture tests enforce who may reference whom.
3. **One pipeline.** Every request passes the same stages, from forwarded headers to the repository. One injected clock decides "today", and one method rounds money.
4. **Errors come in order.** A 400, 404 or 409 is returned alone. Only then are all the 422 rules checked together. Every failure is one RFC 9457 body that names the rule.
5. **A cancellation is a decision.** Five outcomes decide the refund, and the response names the one that applied in `ruleApplied`.
6. **Each test proves something different.** Domain, Services, Api, Architecture, the web suites and Postman each catch a different kind of mistake. Every rule ID is a test trait.
7. **Test first, shipped as evidence.** Outside-in TDD built it and left a trail. The Bicep and the pipeline were built and checked locally and never deployed.

Pick your journey

Developers follow DJ1 to DJ8. Servicers follow SJ1 to SJ8.

Each journey is a list of steps. A step opens with what you decide and what you get back, then shows the real command and its output. The developer's journey is also the assessor's: it answers whether the brief is met and how to check it.

Journey	You are	You start by	You end with
Developer joining (DJ1 to DJ8)	Someone who clones the repository, or assesses it	Reading what the project is and building it	Knowing where a rule lives and how to change it safely
Servicer driving the API (SJ1 to SJ8)	Someone who calls the API with curl or Postman	A first call to a healthy API	Reading any error and running the whole collection

Journey 1: a developer joining the project

Clone it, run it, prove it, then follow a request and a rule through the code.

The commands are for a POSIX shell: Git Bash on Windows, or bash or zsh elsewhere. Run them from the repository root. Clone to a short path on Windows (see Troubleshooting).

Get it running (DJ1 to DJ4)

DJ1 · What you are joining

> **You decide** Whether the brief is met, and which of the assumptions you accept.

> **The repository gives you** The brief's four operations and its could-haves, every rule with an ID and where it is written, the assumptions table, and what is out of scope.

A REST API that sells, retrieves, cancels, renews and records claims on home insurance policies. It is the take-home for Uinsure's senior engineer role. The brief asked for sell, retrieve, cancel and renew; all its MUST, SHOULD and COULD rules are built, plus a cancellation quote and claims. Each rule has an ID and one agreed interpretation in `docs/requirements.md`, one row per ID.

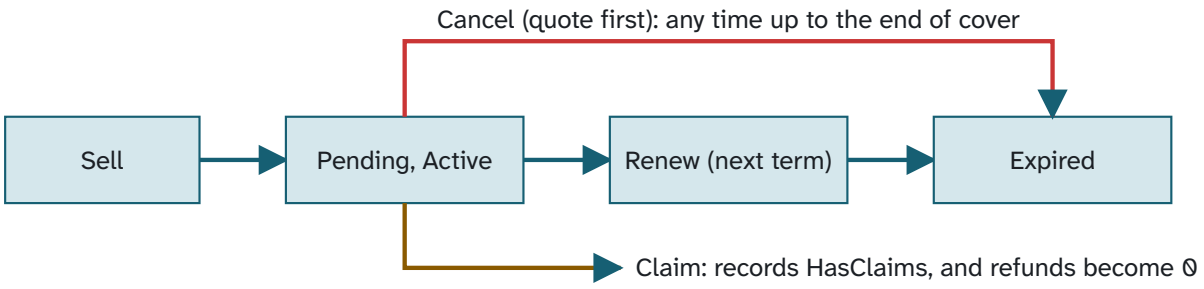


Figure 1. The operations on a policy's life. Status is derived from today, never stored.

Operation	Route	Rules
Sell	`POST /policies`	`SELL-1` to `SELL-7`
Retrieve	`GET /policies/{reference}`	`GET-1`
Cancel, and quote	`POST …/cancellation`, `GET …/cancellation-quote`	`CANCEL-1a` to `CANCEL-7`, `COULD-1`
Renew	`POST …/renewals`	`RENEW-1/2` to `RENEW-6`, `COULD-3`
Record a claim	`POST …/claims`	`COULD-2`, `COULD-2a` to `COULD-2c`

All six routes sit on one sealed controller.

`src/Uinsure.Policies.Api/Controllers/PoliciesController.cs:13`

```
[ApiController]
[Route("policies")]
public sealed class PoliciesController(IPolicyService service) : ControllerBase
{
    [HttpPost]
    [EnableRateLimiting("writes")]
    public async Task<ActionResult<PolicyResponse>> Sell(SellPolicyRequest request,
CancellationToken cancellationToken)
```

Where the brief was unclear, the interpretation is written down, not guessed. The full list is the assumptions section of the README (README.md:181). The ones that change what you see:

Question in the brief	Taken as	Rule
Who chooses the reference?	The client. It is trimmed and upper-cased, and `abc-1` is `ABC-1`	`SELL-1`
"Up to 60 days in advance"	Start date from today to today + 60, inclusive. No backdating	`SELL-2`
One year in length	End = start + 1 year − 1 day. A 29 Feb start ends on 28 Feb	`SELL-3`
The cooling-off period	Runs to start + 14 days, inclusive. After it, the refund is pro rata over the term's real days (365 or 366)	`CANCEL-1b` `CANCEL-1c`
What is a renewal?	A new term on the same policy, starting the day after the end date	`RENEW-4`
Claims	Any claim sets `hasClaims` and makes every later refund £0	`COULD-2`

> **Out of scope**
>
> 1. Authentication. The reference is the only key, so anyone who guesses it can read the policy or post a claim.
> 2. Pricing (the client sends every amount) and real payment processing.
> 3. Live deployment. The Azure files are evidence (DJ8).

The known limits, including the security scope, are in `docs/design.md` and the README's "Known limits".

You should see: The assumptions table above: one agreed reading for every place the brief left a choice, each with its rule ID.

DJ2 · Get it running

> **You decide** Which launch profile to use: `http` in a terminal, `https` for HTTPS, or Visual Studio, alone or with the web app.

> **The repository gives you** A healthy API on `http://localhost:5080` with SQLite and no set-up, its health checks, the OpenAPI description and the Scalar page.

1. Check the SDK. `global.json` pins `10.0.100` with `rollForward` set to `latestFeature`, so any 10.0.1xx or later feature band works.

   ```
   dotnet --version
   ```

   You should see: The installed version:

   ```
   10.0.402
   ```

2. Build with warnings as errors, as the pre-commit hook does.

```
dotnet build Uinsure.Policies.slnx -warnaserror
```

You should see: No warnings and no errors. The last lines of the run:

```
Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:01.49
```

3. Run the API on port 5080 with the `http` launch profile (Development, so SQLite). Leave it running.

```
dotnet run --project src/Uinsure.Policies.Api --launch-profile http
```

You should see: The start-up lines:

```
      Now listening on: http://localhost:5080
      Application started. Press Ctrl+C to shut down.
      Hosting environment: Development
      Content root path: <repo>\src\Uinsure.Policies.Api
```

4. In a second terminal, check health. `/health/live` runs no checks. `/health/ready` runs the checks tagged `ready`, which here is the database. In Development only, OpenAPI is at `/openapi/v1.json` and the Scalar page at `/scalar/v1`.

```
curl -s -i http://localhost:5080/health/live
curl -s -o /dev/null -w "openapi status %{http_code}\n"
http://localhost:5080/openapi/v1.json
curl -s -o /dev/null -w "scalar status %{http_code}\n"
http://localhost:5080/scalar/v1
```

You should see: Health answers 200 with a plain-text body, then the two pages answer 200:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Cache-Control: no-store, no-cache
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Pragma: no-cache

Healthy
```

```
openapi status 200
scalar status 200
```

In Visual Studio

1. Open `Uinsure.Policies.slnx`. Set **Uinsure.Policies.Api** as the startup project, choose the **http** launch profile and press Ctrl+F5. A browser opens on `/health/live` and shows `Healthy`.

2. To start the API and the web app (Docket) together, choose the **API and web** profile, defined in `Uinsure.Policies.slnLaunch`. It starts the `http` profile of the API and Docket's Vite dev server. Docket needs Node 22.22 or later, and `npm --prefix src/Uinsure.Policies.Web ci` once.

Only one process can hold port 5080, so stop a terminal run first. The UI guide covers Docket itself.

PLACEHOLDER: screenshot to be taken by hand

Visual Studio: startup project and the launch profile list (figure 2)

Figure 2. Visual Studio's startup toolbar with Uinsure.Policies.Api and the profile list (http, https, API and web). Placeholder: the real image is a hand capture, and its red boxes are measured from its pixels once it exists.

You should see: Health and the two pages answer 200 on port 5080.

DJ3 · Choose a database

> **You decide** SQLite locally, or SQL Server in Azure. It is one configuration key.

> **The repository gives you** The two configuration keys, a schema created at start-up on SQLite, and an idempotent migration script for SQL Server.

One `PoliciesDbContext` serves both. `Database:Provider` (`Sqlite` or `SqlServer`) chooses the provider and `ConnectionStrings:Policies` holds the connection. The development file sets SQLite and a database file per checkout (`policies.development.db`, ignored by version control).

`src/Uinsure.Policies.Persistence/PersistenceServiceCollectionExtensions.cs:18`

```
        services.AddDbContext<PoliciesDbContext>((serviceProvider, options) =>
        {
            var connectionString = configuration.GetConnectionString("Policies");
            _ = serviceProvider.GetRequiredService<IOptions<DatabaseOptions>>
().Value.ProviderKind switch
            {
                DatabaseProvider.Sqlite => options.UseSqlite(connectionString),
                DatabaseProvider.SqlServer =>
options.UseSqlServer(connectionString),
                _ => throw new InvalidOperationException("Unknown database
provider."),
            };
        });
```

src/Uinsure.Policies.Api/appsettings.Development.json:1

```
{
  "Database": {
    "Provider": "Sqlite"
  },
  "ConnectionStrings": {
    "Policies": "Data Source=policies.development.db"
  },
```

With SQLite, the schema is created from the model at start-up (`EnsureCreated`, src/Uinsure.Policies.Persistence/PersistenceServiceCollectionExtensions.cs:37). The app never calls `Migrate()`. The migration, `InitialCreate`, is for SQL Server only, and the pipeline applies it as a script.

1. Generate the idempotent script, as the pre-push hook does.

   ```
   dotnet ef migrations script --idempotent --project
   src/Uinsure.Policies.Persistence --startup-project src/Uinsure.Policies.Api -o
   migrate.sql
   ```

   You should see: The tool wrote the script and printed one warning about its own version:

   ```
   Build started...
   Build succeeded.
   The Entity Framework tools version '9.0.9' is older than that of the runtime
   '10.0.13'. Update the tools for the latest features and bug fixes. See
   https://aka.ms/AAc1fbw for more information.
   ```

2. Count its lines, then read the first ones.

   ```
   wc -l < migrate.sql
   head -n 9 migrate.sql
   ```

   You should see: A T-SQL script of this many lines, which starts by creating the migrations history table if it is missing:

   ```
   242
   ```

   ```
   IF OBJECT_ID(N'[__EFMigrationsHistory]') IS NULL
   BEGIN
       CREATE TABLE [__EFMigrationsHistory] (
           [MigrationId] nvarchar(150) NOT NULL,
           [ProductVersion] nvarchar(32) NOT NULL,
           CONSTRAINT [PK___EFMigrationsHistory] PRIMARY KEY ([MigrationId])
       );
   END;
   GO
   ```

> **No SQL Server here**
> No test runs against SQL Server, and this guide ran none. What is shown for it is the configuration above and the generated script. The README says the same under "Known limits".

Persistence has a few rules of its own. The aggregate is loaded whole. `Reference` has a unique index, so even a race on **SELL-1** ends in a 409. `Version` is a concurrency token, set anew on every save of a changed policy, so two writes on one policy give a 409 `/problems/concurrency`. Money is stored with precision 18,2 and enums as strings. The code never orders or sums money in SQL, because SQLite stores decimals as text.

`src/Uinsure.Policies.Persistence/Configurations/PolicyConfiguration.cs:14`

```
        // SELL-1: the stored reference is already normalised, so a plain unique
index is case-insensitive in effect.
        builder.Property(p => p.Reference).HasMaxLength(20).IsRequired();
        builder.HasIndex(p => p.Reference).IsUnique();

        builder.Property(p => p.Type).HasConversion<string>().HasMaxLength(20);
        builder.Property(p => p.Version).IsConcurrencyToken();
```

You should see: The script's line count and its first lines (above), and the two keys that choose the provider.

DJ4 · Prove it works (run the tests yourself)

> **You decide** Which tests to run (all, one project, one rule) and with which tool: the command line, Visual Studio, the Postman CLI or app (SJ8), or the web suites.

> **The repository gives you** Totals per project, a run of one rule, rule coverage, the web test totals and the harness self-test, and a clear account of what each project can and cannot catch.

Domain.Tests (173)	Every rule at both sides of its boundary, with today passed in
Services.Tests (65)	Orchestration and UkCalendar, including the BST boundary
Api.Tests (335)	The whole path over HTTP: SQLite in memory, a fake clock
Architecture.Tests (79)	Layering, banned APIs, and a trait for every rule ID
Web suites (Vitest 421)	Vitest units; Playwright screens against a live API
Postman collection	The running API, called as a client calls it

Figure 3. What each suite proves, from the rule outward to the running API.

From the command line (MAN-4)

1. Run everything.

```
dotnet test --solution Uinsure.Policies.slnx
```

You should see: Four projects pass, then the totals:

```
Test run summary: Passed!

<repo>\tests\Uinsure.Policies.Services.Tests\bin\Debug\net10.0\Uinsure.Policies.
Services.Tests.dll (net10.0|x64) passed (831ms)

<repo>\tests\Uinsure.Policies.Api.Tests\bin\Debug\net10.0\Uinsure.Policies.Api.T
ests.dll (net10.0|x64) passed (7s 682ms)

<repo>\tests\Uinsure.Policies.Domain.Tests\bin\Debug\net10.0\Uinsure.Policies.Do
main.Tests.dll (net10.0|x64) passed (953ms)

<repo>\tests\Uinsure.Policies.Architecture.Tests\bin\Debug\net10.0\Uinsure.Polic
ies.Architecture.Tests.dll (net10.0|x64) passed (1s 418ms)

  total: 652
  failed: 0
  succeeded: 652
  skipped: 0
  duration: 7s 899ms
```

2. Run one project.

```
dotnet test --project tests/Uinsure.Policies.Domain.Tests
```

You should see: Only that project's totals. The other three gave 65 (Services), 335 (Api) and 79 (Architecture), each with none failed:

```
Test run summary: Passed!
  total: 173
  failed: 0
  succeeded: 173
  skipped: 0
  duration: 993ms
```

3. Run one rule by its trait.

```
dotnet test --solution Uinsure.Policies.slnx -- --filter-trait "Rule=CANCEL-1b"
```

You should see: Eight tests across the four projects, all passed:

```
Test run summary: Passed!

<repo>\tests\Uinsure.Policies.Services.Tests\bin\Debug\net10.0\Uinsure.Policies.
Services.Tests.dll (net10.0|x64) passed (632ms)

<repo>\tests\Uinsure.Policies.Api.Tests\bin\Debug\net10.0\Uinsure.Policies.Api.T
ests.dll (net10.0|x64) passed (2s 266ms)

<repo>\tests\Uinsure.Policies.Domain.Tests\bin\Debug\net10.0\Uinsure.Policies.Do
main.Tests.dll (net10.0|x64) passed (874ms)

<repo>\tests\Uinsure.Policies.Architecture.Tests\bin\Debug\net10.0\Uinsure.Polic
ies.Architecture.Tests.dll (net10.0|x64) passed (627ms)

  total: 8
  failed: 0
  succeeded: 8
  skipped: 0
  duration: 2s 482ms
```

4. Run the rule-coverage tests alone.

```
dotnet test --project tests/Uinsure.Policies.Architecture.Tests -- --filter-
class "*RuleCoverageTests"
```

You should see: Eleven coverage tests passed:

```
Test run summary: Passed!
  total: 11
  failed: 0
  succeeded: 11
  skipped: 0
  duration: 1s 162ms
```

> **Careful**
> The filter flag comes after `--`, and the option is `--filter-trait` (xUnit v3 on Microsoft.Testing.Platform). The older `--filter "Rule=…"` form is for VSTest and is not what this solution runs on.

Each test carries `[Trait("Rule", "<id>")]`. `RuleCoverageTests` fails if a rule ID in `docs/requirements.md` lacks a unit trait or an integration trait, unless it is listed as pending.

tests/Uinsure.Policies.Api.Tests/CancelCoolingOffTests.cs:22

```
    // 2026-10-17 is both the start date and today (CANCEL-3 allows today): 1 day
used, 289.40 back to the Cheque.
    [Fact]
    [Trait("Rule", "CANCEL-1b")]
    [Trait("Rule", "CANCEL-2")]
    [Trait("Rule", "CANCEL-3")]
    public async Task Cancel_on_the_start_date_refunds_in_full_to_the_Cheque_paid()
    {
        const string reference = "CAN1B-START";
        var sold = new SellPolicyRequestBuilder(reference)

.WithStartDate(TestClock.Today).WithAmount(289.40m).WithPaymentMethod("Cheque").Bui
ld();
```

In Visual Studio (MAN-3)

1. Open `Uinsure.Policies.slnx`. For every folder on disk, use **Switch Views** → **Folder View**.
2. Open **Test** → **Test Explorer** and choose **Run All Tests** (Ctrl+R, A). The tests group by project, and passing tests show a green tick.
3. Run one project, class or test: right-click it and choose **Run**. To run one rule, use **Group By** → **Traits**: the `Rule` trait lists tests by rule ID, and you run the group.
4. Debug a test: right-click it and choose **Debug**. To read a failure, select the test: the detail pane shows the message and stack, and the assertion line is a link.

PLACEHOLDER: screenshot to be taken by hand

Visual Studio: Test Explorer, grouped by Rule trait (figure 4)

Figure 4. Test Explorer grouped by the Rule trait. Placeholder: a hand capture.

These Visual Studio steps come from the README and were not run for this guide.

The web suites and the harness self-test (MAN-5, MAN-6)

1. Run Docket's unit tests (Vitest). They need nothing running.

   ```
   npm --prefix src/Uinsure.Policies.Web run test
   ```

   You should see: The test files and tests all pass:

   ```
    Test Files  63 passed (63)
         Tests  421 passed (421)
      Duration  52.58s (tests 82%, environment 13%, import 5%, transform 1%)
   ```

2. Playwright, Docket's end-to-end suite, starts its own API and app: `npm --prefix src/Uinsure.Policies.Web run e2e`. It takes about an hour, so it was not run for this guide. The UI guide gives its commands in full, with the single-spec, headed and report options.

3. Run the harness self-test, which checks the repository's own guard hooks.

   ```
   node --test ".claude/hooks/__tests__/*.test.mjs"
   ```

   You should see: Every test passes. It is the slowest check here apart from Playwright:

   ```
   1..234
   # tests 234
   # suites 0
   # pass 234
   # fail 0
   # cancelled 0
   # skipped 0
   # todo 0
   # duration_ms 57214.6363
   ```

Which runner, when (MAN-7)

Runner	Tests	Use when	Command or click path	Needs first	Measured
`dotnet test`	All 652 .NET tests	Before any commit; in CI	`dotnet test --solution Uinsure.Policies.slnx`	Nothing	7.9 s test time
One rule by trait	The tests of one rule ID	Working on one rule	`… -- --filter-trait "Rule=CANCEL-1b"`	Nothing	2.5 s (8 tests)
Vitest	Docket's units	Changing the web app	`npm --prefix src/Uinsure.Policies.Web run test`	Nothing	52.6 s
Playwright	Docket's screens against a live API	Before a UI change lands	`npm --prefix src/Uinsure.Policies.Web run e2e`	Nothing (it starts its own)	Not run for this guide
Postman CLI	The collection against the live API	Check a running build as a client	`postman collection run …`	The API on 5080	1867 ms for the F2 folder (F8 receipt)
Postman app	The same collection	Explore, edit, run one request	Import, then ⋯ → Run collection	The API on 5080	Not measured (hand run)

Runner	Tests	Use when	Command or click path	Needs first	Measured
Visual Studio	The .NET tests	Debug a test, group by rule	Test → Test Explorer	Nothing	Not measured (hand run)
Harness self-test	The guard-hook tests	After changing a hook	`node --test ".claude/hooks/__tests__/*.test.mjs"`	Node	57.2 s

You should see: All four .NET projects pass, then the web unit tests and the self-test, with the totals above. The Postman CLI run is SJ8.

Understand it and change it (DJ5 to DJ8)

DJ5 · Follow a request

> **You decide** Which layer to read first: the controller (what HTTP promises), the service (what happens) or the domain (the rules).

> **The repository gives you** Four projects and who may reference whom, the pipeline in the order `Program.cs` builds it, and how logging and health work.

The domain holds every rule and uses only the .NET base library. Services orchestrate: load, compute today, call the domain, save. Persistence stores. The Api maps HTTP to a service call and back. Architecture tests enforce the arrows.

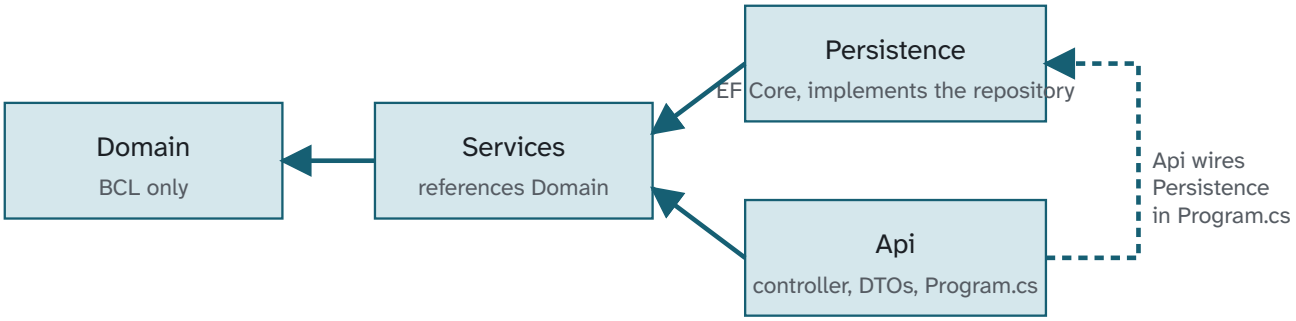


Figure 5. Who may reference whom. An arrow points at what is referenced.

Project	May reference	Must not
Domain	The .NET base library only	Any package, any Uinsure project, `TimeProvider`
Services	Domain	EF Core, ASP.NET Core, Persistence, Api
Persistence	Services, Domain, EF Core	ASP.NET Core, Api
Api	Services, Persistence, Domain	Any test project. Only `Program` touches Persistence

tests/Uinsure.Policies.Architecture.Tests/LayeringTests.cs:11

```
    [Fact]
    public void
Domain_types_depend_on_no_Microsoft_namespace_and_not_on_TimeProvider()
    {
        var result = Types.InAssembly(SourceAssemblies.Domain)
            .Should().NotHaveDependencyOnAny("Microsoft", "System.TimeProvider")
            .GetResult();

        Assert.Empty(ArchResults.FailingTypeNamesIn(result,
"Uinsure.Policies.Domain"));
    }
```

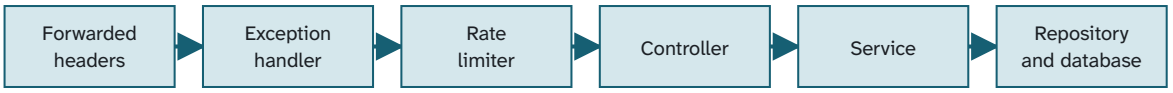


Figure 6. The request pipeline, in the order `Program.cs` builds it.

src/Uinsure.Policies.Api/Program.cs:123

```
app.UseRateLimiter();
app.UseAuthorization();

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.MapControllers();
app.MapHealthChecks("/health/live", new() { Predicate = _ => false
}).DisableRateLimiting();
app.MapHealthChecks("/health/ready", new() { Predicate = check =>
check.Tags.Contains("ready") }).DisableRateLimiting();
```

Before those lines, `Program.cs` applies forwarded headers and the exception handler (src/Uinsure.Policies.Api/Program.cs:90). Outside Development it rejects plain HTTP with 400 `/problems/https-required` instead of redirecting (src/Uinsure.Policies.Api/Program.cs:96). Input is strict: unknown JSON fields and query keys are 400s, and the body is capped at 65,536 bytes. The controller turns a service `Result` into a response, and the service does the orchestration:

src/Uinsure.Policies.Services/PolicyService.cs:12

```
        var today = UkCalendar.Today(clock);

        // SELL-1: a duplicate is a 409 returned alone, before any 422 rule is
evaluated.
        if (await repository.GetByReference(command.Reference, ct) is not null)
        {
            return
Result<PolicyDetails>.Failure(DuplicateReference(command.Reference));
        }
```

Logging and health. Logs use message templates, with the reference, rule IDs, outcome and trace ID. They never carry names, dates of birth, addresses, postcodes or request bodies. Application Insights is wired in only when `APPLICATIONINSIGHTS_CONNECTION_STRING` is set (src/Uinsure.Policies.Api/Program.cs:81). `/health/live` runs no checks; `/health/ready` runs the database check, which is registered where the database is (src/Uinsure.Policies.Persistence/PersistenceServiceCollectionExtensions.cs:31).

You should see: Figures 5 and 6 and the three extracts: the layers, the pipeline in `Program.cs` order, and the service's load, today, domain, save.

DJ6 · Follow a rule

> **You decide** Which rule to follow. CANCEL-1b is followed here, and any other ID works the same way.

> **The repository gives you** The requirement, the traited tests and the code side by side, and the two places that decide "today" and rounding.

1. **Requirement.** docs/requirements.md:77: "On or after the start date, within the 14-day cooling-off → full refund", interpreted as start + 14 days, inclusive.
2. **Test.** tests/Uinsure.Policies.Api.Tests/CancelCoolingOffTests.cs:22 sells a policy, cancels it on the start date and asserts the refund (extract in DJ4). The unit tests sit in `Domain.Tests`.
3. **Code.** src/Uinsure.Policies.Domain/Rules/RefundCalculator.cs:40. One pure method serves both the cancellation and its quote.

```
        // CANCEL-1b: cooling-off runs to start + 14 days, inclusive.
        if (cancelOn <= term.Start.AddDays(CoolingOffDays))
        {
            return (RuleIds.Cancel1b, term.Amount);
        }

        // CANCEL-1c: the cancellation day counts as used; divide by the term's
actual days (365 or 366).
        return (RuleIds.Cancel1c, term.Amount * unusedDays / termDays);
```

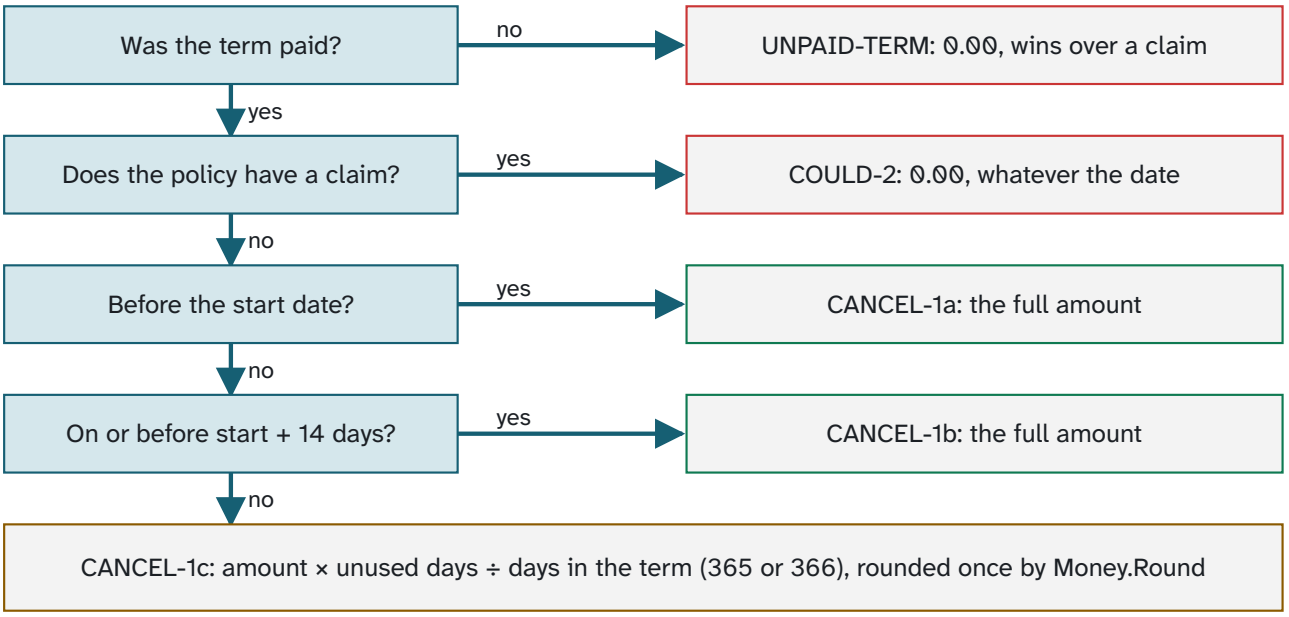


Figure 7. How one term's refund is decided, in the order `RefundCalculator` checks. Each term is decided on its own, and the answer names the rule in `ruleApplied`.

Run it: the trait filter in DJ4 ran the 8 tests of this rule, all green.

Time and money. "Today" has one source. Services read the injected `TimeProvider` and convert to Europe/London once; the domain receives a `DateOnly`. An analyser (`BannedSymbols.txt`) turns `DateTime.Now`, `UtcNow` and `Today` into build errors. Money is rounded only in `Money.Round`, and an architecture test allows the analyser suppression in that file alone.

`src/Uinsure.Policies.Services/UkCalendar.cs:5`

```
    private static readonly TimeZoneInfo London =
TimeZoneInfo.FindSystemTimeZoneById("Europe/London");

    public static DateOnly Today(TimeProvider clock) =>
        DateOnly.FromDateTime(TimeZoneInfo.ConvertTime(clock.GetUtcNow(),
London).DateTime);
```

`src/Uinsure.Policies.Domain/Common/Money.cs:5`

```
    // CANCEL-6: 2 dp, half away from zero. .NET's default is banker's rounding, so
this is the one place it is chosen.
#pragma warning disable RS0030 // The only permitted call to decimal rounding
(architecture test A11).
    public static decimal Round(decimal amount) => Math.Round(amount, 2,
MidpointRounding.AwayFromZero);
#pragma warning restore RS0030
```

The 29 February case is in the domain too:

src/Uinsure.Policies.Domain/Policies/Term.cs:23

```
    // SELL-3: start + 1 year - 1 day; a 29 Feb start has no anniversary, so it
ends on 28 Feb.
    public static DateOnly EndFor(DateOnly start) =>
        start is { Month: 2, Day: 29 }
            ? new DateOnly(start.Year + 1, 2, 28)
            : start.AddYears(1).AddDays(-1);
```

You should see: The requirement row, the traited test and the calculator side by side, and Figure 7 showing every outcome that code can return.

DJ7 · Change it safely

> **You decide** What to add: a rule or an endpoint.

> **The repository gives you** An order of work that keeps tests first, and two git hooks that refuse a bad commit or push.

1. Write the row in `docs/requirements.md` with an ID and its interpretation.
2. Write a red test with `[Trait("Rule", "<id>")]`, at HTTP level, then at domain level.
3. Add the domain method and its `RuleIds` entry, then the `ProblemCatalog` entry for its status and title.
4. Add the controller action. Run the architecture tests; they fail if a layer arrow is wrong.

Pre-commit refuses commits on a locked `main`, scans the staged files for private content, and builds with warnings as errors. **Pre-push** refuses pushes to `main`, then builds in Release, runs the whole suite, generates the idempotent migration script, and runs the harness self-test.

.githooks/pre-push:25

```
if [ -f Uinsure.Policies.slnx ]; then
  dotnet build Uinsure.Policies.slnx -c Release -warnaserror -nologo -v quiet || {
echo "pre-push: build failed" >&2; exit 1; }
  dotnet test --solution Uinsure.Policies.slnx -c Release --no-build || { echo
"pre-push: tests failed" >&2; exit 1; }
  out="${TMPDIR:-${TEMP:-/tmp}}/uinsure-migrate.sql"
  dotnet tool restore >/dev/null && dotnet ef migrations script --idempotent \
    --project src/Uinsure.Policies.Persistence --startup-project
src/Uinsure.Policies.Api -o "$out" \
    || { echo "pre-push: idempotent migration script failed" >&2; exit 1; }
```

1. Run the pre-push hook yourself, as git runs it, with no refs on standard input.

```
sh .githooks/pre-push < /dev/null; echo "pre-push exit $?"
```

You should see: The last lines of the run, ending in exit 0:

```
<repo>\tests\Uinsure.Policies.Architecture.Tests\bin\Release\net10.0\Uinsure.Pol
icies.Architecture.Tests.dll (net10.0|x64) passed (1s 513ms)

<repo>\tests\Uinsure.Policies.Domain.Tests\bin\Release\net10.0\Uinsure.Policies.
Domain.Tests.dll (net10.0|x64) passed (1s 007ms)

  total: 652
  failed: 0
  succeeded: 652
  skipped: 0
  duration: 3s 918ms
Build started...
Build succeeded.
The Entity Framework tools version '9.0.9' is older than that of the runtime
'10.0.13'. Update the tools for the latest features and bug fixes. See
https://aka.ms/AAc1fbw for more information.
pre-push exit 0
```

You should see: The hook's own output: the Release test run passes and the hook exits 0, so a push would go ahead.

DJ8 · How it was built, and how it would ship

> **You decide** How much to trust the trail, and whether the Azure files are enough evidence.

> **The repository gives you** The test-first loop and the trail it leaves, in one page that links to Docket for the full story. Then the Bicep and the pipeline, and the reason nothing is deployed.

AI agents in Claude Code built this repository inside a harness, with a human deciding at fixed points. Every rule in the brief got an ID; every feature is a node in a plan; every node leaves a trail of commits, verdicts and findings.

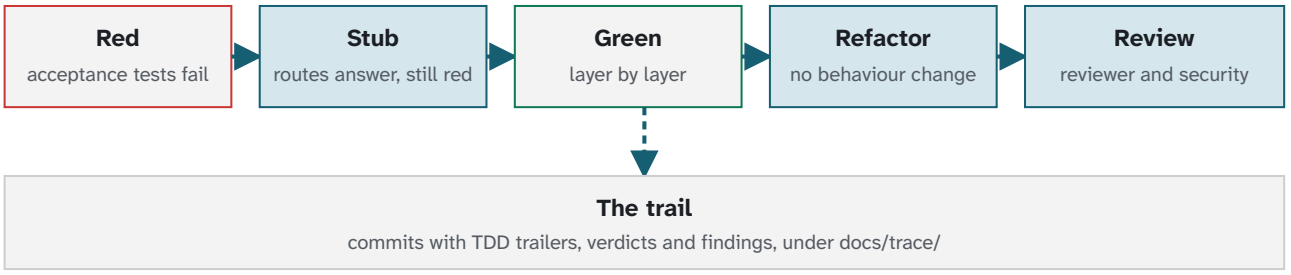


Figure 8. The loop that built each node, and the trail it leaves behind.

Idea	What it means here
Outside-in TDD	Red acceptance tests first, then a stub controller, then domain, services, persistence and API turned green layer by layer. Commits carry a `TDD:` trailer (`red` , `stub` , `green` , `refactor`).
The plan DAG	Features are nodes with tasks in `docs/plans/active/build.dag.yaml` . Status changes only by commit.
Six agents	Tester, builder, reviewer, security, investigator and git. The agents that review cannot write, and the ones that write do not judge their own work. The orchestrator runs every gate itself.
The trail	`docs/trace/build/<node>/` : activity, verdicts, findings and a walkthrough.

A finding sends a node back through the failure route: diagnose, write a new red test, then fix. The third failure stops for a written investigation. This command counts, per node, the findings files and the verdict files:

```
for n in $(ls docs/trace/build); do echo "$n findings=$(ls
docs/trace/build/$n/findings 2>/dev/null | wc -l) verdicts=$(ls
docs/trace/build/$n/verdicts 2>/dev/null | wc -l)"; done
```

You should see: One line per node of the build plan:

```
F0a findings=0 verdicts=0
F0b findings=2 verdicts=5
F1 findings=3 verdicts=4
F1b findings=1 verdicts=4
F2 findings=2 verdicts=6
F3 findings=1 verdicts=6
F4 findings=0 verdicts=6
F5 findings=2 verdicts=12
F6 findings=4 verdicts=8
F7 findings=4 verdicts=4
F8 findings=3 verdicts=6
```

The full story, with each node's walkthrough and every finding, is browsable in Docket, the web app, which the UI guide covers.

Shipping: Bicep and the pipeline

> **Evidence only**
> They show how this API would reach Azure. Nothing here was connected to Azure or Azure DevOps, and `az` was never run. No service connection, environment or approval exists, and no what-if was run, because it needs a signed-in subscription. The repository's real gate is its local git hooks.

`infrastructure/main.bicep` works at subscription scope. It creates the resource group `rg-policies-<env>` and calls five modules. Three parameter files give dev, test and prod.

Module	Creates
`modules/monitoring.bicep`	Log Analytics workspace, Application Insights
`modules/network.bicep`	Virtual network, two network security groups
`modules/sql.bicep`	Azure SQL server and database, private endpoint, private DNS
`modules/appservice.bicep`	Managed identity, Linux App Service plan, the web app
`modules/staticwebapp.bicep`	The Static Web App for Docket, linked to the App Service as its backend

1. Build the template locally.

   ```
   bicep --version
   bicep build infrastructure/main.bicep --outfile main.json
   wc -l < main.json
   ```

   You should see: The version, then no output from the build (exit code 0), then the compiled template's line count:

   ```
   Bicep CLI version 0.48.1 (cec7951cf6)
   ```

   ```
   905
   ```

There are no secrets: the app signs in to Entra-only Azure SQL with a user-assigned managed identity, so no password exists. Key Vault was considered and rejected, because there is nothing to store. `infrastructure/pipelines/azure-pipelines.yml` triggers on pushes to `main`, with `pr: none`.

Stage	What it does
Build	Build with `-warnaserror`, run the tests, generate the idempotent migration script, publish the app and the infrastructure folder as one artefact
Web	Install, typecheck, lint, unit tests, compile the trail, build and run Playwright against the API; publish the web artefact
`WhatIf_<env>`	Runs unapproved for dev, test and prod, so the approver can read the what-if
`Deploy_<env>`	Runs the infrastructure deployment, applies the migration script, deploys the zip and the web artefact, smoke-tests the site root and `/health/ready`. Prod waits for the approval on the `policies-prod` environment

Why, in six decisions

> **Why**
> Terms, not overwritten dates. The alternative was to move the policy's dates on renewal. Renewal happens while the old term still runs, so overwriting would break a cancellation made in its last 30 days (`RENEW-4`).

> **Why**
> One result type and one status mapping. The alternative was exceptions per rule, with status codes chosen in controllers. A `Result` lets the domain report every broken rule at once, and `ProblemCatalog` is the only place a status is chosen.

> **Why**
> Interfaces only at the layer seams. The alternative was an interface per class, a generic repository and a mediator. Three seams (service, repository, reference generator) are what the tests need; more would be speculation.

> **Why**
> No `DateTime.Now`. The alternative was to read the clock where needed. One injected clock makes every date boundary testable, and the build fails on the banned calls.

> **Why**
> SQLite locally. The alternative was SQL Server in a container. SQLite needs nothing installed, so the reviewer runs the API in a minute; the provider switch is a configuration key.

> **Why**
> Rounding in one method. .NET rounds halves to even by default. The brief's pro-rata refunds need half away from zero, so it is chosen once in `Money.Round` (`CANCEL-6`). And evidence instead of a deployment: the brief put live deployment out of scope, and an unrun pipeline can claim only what a local check proved.

You should see: Figure 8, the trail counts per node, and a Bicep build that exits 0 with no output.

Check your understanding

Question	Answer
DQ1. What does it do, and does it meet the brief?	Sell, retrieve, cancel (with a quote), renew and claims. Every MUST, SHOULD and COULD rule has an ID in `docs/requirements.md`.
DQ3. What is deliberately not done, and is it safe to expose?	Authentication, pricing, payment processing and live deployment. No: there is no authentication, and the security scope names the gaps.
DQ4. How do I build and run it quickly?	SDK 10, `dotnet build`, `dotnet run --project src/Uinsure.Policies.Api --launch-profile http`. SQLite needs no setup.
DQ5. How does the database get its schema?	`EnsureCreated` on SQLite at start-up. A script made from the EF migrations, for SQL Server. The app never calls `Migrate()`.
DQ7. How can I see each rule has a test, and run one rule's tests?	The `Rule` trait and `RuleCoverageTests`; `-- --filter-trait "Rule=<id>"`, or Group By Traits in Test Explorer.

Question	Answer
DQ9. What happens to a request?	The pipeline in `Program.cs`, the controller, the service (load, today, domain, save), the repository.
DQ10. How do I follow a rule?	Requirement row, traited tests, then the domain method: CANCEL-1b is docs/requirements.md:77, tests/Uinsure.Policies.Api.Tests/CancelCoolingOffTests.cs:22, src/Uinsure.Policies.Domain/Rules/RefundCalculator.cs:40.
DQ11/DQ12. How is "today" decided, and money rounded?	`UkCalendar.Today(TimeProvider)` in Europe/London, and only `Money.Round`: 2 dp, away from zero.
DQ14. What do the hooks run?	Pre-commit: privacy scan and build. Pre-push: Release build, full tests, migration script, harness self-test.
DQ15/DQ16. Was it built test-first, and is the Azure part real?	Outside-in TDD, with `TDD:` trailers and a trail under `docs/trace/build/`. The Azure part is evidence only: Bicep built locally, nothing deployed, `az` never run.

Journey 2: a servicer driving the API

Call the API with curl and Postman, and read every answer.

The examples were sent with `curl` to the API started in DJ2, on 2026-10-25, so "today" is 2026-10-25. Run the steps in order: the later ones use the policies the earlier ones create. A `…` inside a command or an output marks text left out of this page; the full text is in `receipts.md`. The commands are for a POSIX shell (Git Bash on Windows). In PowerShell, `curl` is an alias for a different command, so use `curl.exe` and change the quoting.

Sell, look up and cancel (SJ1 to SJ4)

SJ1 · Make a first call

> **You decide** How to call the API (the Postman app, curl or the Scalar page) and the base URL.

> **The API gives you** A healthy API, the Scalar page that lists every route, and a Postman collection that imports with its environment.

1. Start the API (DJ2, step 3) and ask whether it is ready to serve.

   ```
   curl -s -i http://localhost:5080/health/ready
   ```

   You should see: 200 with the body `Healthy`:

   ```
   HTTP/1.1 200 OK
   Content-Type: text/plain
   Cache-Control: no-store, no-cache
   Expires: Thu, 01 Jan 1970 00:00:00 GMT
   Pragma: no-cache

   Healthy
   ```

2. For a page to click through instead, open `http://localhost:5080/scalar/v1` in a browser (Development only). It lists the six routes and sends requests for you.

The Postman app (MAN-1)

1. Install Postman Desktop. The browser version cannot reach `localhost` without the Desktop Agent.
2. Choose **Import** and select both files in `tests/postman/`: `Uinsure.Policies.postman_collection.json` and `local.postman_environment.json`. The collection **Uinsure.Policies** appears with six folders, and an environment named **local**.
3. Pick **local** in the environment dropdown, top right. It holds one variable, `baseUrl = http://localhost:5080`. Start the API.
4. Open any request and choose **Send**, then read the **Test Results** tab. Running a whole folder or the collection is SJ8.

PLACEHOLDER: screenshot to be taken by hand

Postman: collection imported, environment "local" selected (figure 9)

Figure 9. Postman after the import. Placeholder: a hand capture, with its red boxes measured from the real image.

You should see: A `Healthy` answer from the API, and the collection and the **local** environment in Postman's sidebar.

SJ2 · Sell a policy

> **You decide** The reference, start date, amount, payment method, one to three policyholders and the property.

> **The API gives you** A `201` with a `Location` header and the whole policy, with its end date, term and payment calculated. Or the failure: SELL-1 as a 409, SELL-2 as a 422.

`SELL-1` to `SELL-7`. `POST /policies` takes the reference you choose, the type (`Household` or `BuyToLet`), the start date, the amount, `autoRenew`, the payment method (`Card`, `DirectDebit`, `Cheque`), one to three policyholders and the property. The end date is calculated.

1. Sell a policy that starts 2026-11-01.

```
curl -s -i -X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GUIDE-
001","type":"Household","startDate":"2026-11-
01","amount":803.40,"autoRenew":true,"paymentMethod":"Card","policyholders":
[{"firstName":"Ada","lastName":"Lovelace","dateOfBirth":"1985-12-
10"}],"property":{"addressLine1":"1 Example
Street","addressLine2":"Exampleton","postcode":"EX1 1AA"}}'
```

   You should see: 201 with a `Location` header and the whole policy:

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /policies/GUIDE-001

{"reference":"GUIDE-
001","type":"Household","status":"Pending","currentTermNumber":1,"startDate":"20
26-11-01","endDate":"2027-10-
31","amount":803.40,"hasClaims":false,"autoRenew":true,"policyholders":
[{"firstName":"Ada","lastName":"Lovelace","dateOfBirth":"1985-12-
10"}],"property":{"addressLine1":"1 Example
Street","addressLine2":"Exampleton","addressLine3":null,"postcode":"EX1
1AA"},"terms":[{"termNumber":1,"startDate":"2026-11-01","endDate":"2027-10-
31","amount":803.40,"paymentReference":"PAY-
3e6b82d3846e4b3784ea7086a0066122"}],"payments":[{"reference":"PAY-
3e6b82d3846e4b3784ea7086a0066122","type":"Card","amount":803.40}],"refunds":
[],"claims":[],"cancellation":null}
```

2. Send the same reference again.

```
curl -s -i -X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '…the same body…'
```

   You should see: 409: `SELL-1`. A duplicate is refused, in any letter case, before any 422 rule is looked at.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/sell-1","title":"Policy reference already
exists","status":409,"detail":"A policy with reference 'GUIDE-001' already
exists.","instance":"/policies","traceId":"00-c66f182bb0580d5e06ac8fa51900e081-
31db072268fc0926-00","ruleId":"SELL-1","reference":"GUIDE-001"}
```

3. Send a sale with a start date in the past and an amount of `0`.

```
curl -s -i -X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GUIDE-
003","type":"Household","startDate":"2026-10-01","amount":0,…}'
```

You should see: 422 with *both* broken rules in `violations[]`. `ruleId` is the first of them in requirements order. The sale window is today to today + 60:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/sell-2","title":"Start date outside the sale
window","status":422,"detail":"2 rules
failed.","instance":"/policies","traceId":"00-dd425d7c34176da3dd7e592810d2ed7d-
55d896e22b406256-00","ruleId":"SELL-2","violations":[{"ruleId":"SELL-
2","detail":"The start date must be between 2026-10-25 and 2026-12-24."},
{"ruleId":"SELL-7","detail":"The policy amount must be greater than 0."}]}
```

4. Send a body with only `{"reference":"GU"}`.

```
curl -s -i -X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GU"}'
```

You should see: 400: a shape error. It has an `errors` map and no `ruleId`:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json; charset=utf-8

{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or
more validation errors occurred.","status":400,"instance":"/policies","errors":
{"type":["The Type field is required."],"amount":["The Amount field is
required."],"property":["The Property field is required."],"autoRenew":["The
AutoRenew field is required."],"startDate":["The StartDate field is
required."],"paymentMethod":["The PaymentMethod field is
required."],"policyholders":["The Policyholders field is
required."]},"traceId":"00-c2886f2666a7438187caa51ab61cddfb-48e9f97bb9de687a-
00"}
```

The later steps also use a second policy, GUIDE-002 (1,200.00 by Direct Debit, auto-renew off), that starts today. Sell it now. It answers 201 with the same shape as the first sale.

```
curl -s -i -X POST http://localhost:5080/policies -H "Content-Type:
application/json" -d '{"reference":"GUIDE-002","type":"BuyToLet","startDate":"2026-
10-
25","amount":1200.00,"autoRenew":false,"paymentMethod":"DirectDebit","policyholders
":[{"firstName":"Grace","lastName":"Hopper","dateOfBirth":"1990-12-
09"}],"property":{"addressLine1":"2 Sample Road","postcode":"EX2 2BB"}}'
```

Failure	Status	Rule
Reference already used (any case)	409	SELL-1
Start date not between today and today + 60	422	SELL-2
Not 1 to 3 policyholders; one is under 16 on the start date	422	SELL-4 SELL-5
Amount 0 or below	422	SELL-7
Missing field, unknown field, wrong type, reference not 3 to 20 letters, digits or hyphens	400	none (shape)

In Postman: folder **F1 Sell**, request `SELL-1 · duplicate reference → 409`.

You should see: The 201 body above: `status` is `Pending` because the start date is in the future, and `endDate` is 2027-10-31.

SJ3 · Look it up

> **You decide** Which reference to read.

> **The API gives you** The policy with its derived status and current term, or a 404.

`GET-1`. `GET /policies/{reference}` returns the policy, its terms, policyholders, property, payments, refunds and claims. The reference in the path is trimmed and upper-cased, so a lower-case one finds it. The policy-level dates and amount describe the current term.

Status	When
`Pending`	Today is before the first start date
`Active`	Today falls inside a term
`Expired`	Today is after the latest term's end
`Cancelled`	There is a cancellation. It overrides the rest

1. Read it back, using a lower-case reference.

```
curl -s -i http://localhost:5080/policies/guide-001
```

You should see: 200 with the same body as the sale response:

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-
001","type":"Household","status":"Pending","currentTermNumber":1,…,"cancellation
":null}
```

2. Ask for a reference that does not exist.

```
curl -s -i http://localhost:5080/policies/NO-SUCH-POLICY
```

You should see: 404: `GET-1`.

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/get-1","title":"Policy not found","status":404,"detail":"No policy with reference 'NO-SUCH-POLICY'.","instance":"/policies/NO-SUCH-POLICY","traceId":"00-f6e3a7d7f44869b8afdd1f8d48d80df5-6cb564719bf094d1-00","ruleId":"GET-1","reference":"NO-SUCH-POLICY"}
```

3. Add a query key the route does not bind.

```
curl -s -i "http://localhost:5080/policies/GUIDE-001?reference=X"
```

You should see: 400. Unknown query keys are refused, and a key named like a route parameter counts as unknown:

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json; charset=utf-8

{"type":"https://tools.ietf.org/html/rfc9110#section-15.5.1","title":"One or more validation errors occurred.","status":400,"instance":"/policies/GUIDE-001","errors":{"reference":["Unknown query parameter."]},"traceId":"00-313edcd565faceb8728a5f80b0f095fe-e0d5b258e3099dbc-00"}
```

In Postman: folder **F2 Retrieve**, request `GET-1 · unknown reference → 404`.

You should see: The 200 body, with `status` `Pending` and `currentTermNumber` 1, and the two refusals beneath it.

SJ4 · Quote, then cancel

> **You decide** The date to cancel on, and whether to go ahead once you have seen the refund.

> **The API gives you** The refund for each term and the rule that decided it (`ruleApplied`), first as a read-only quote and then as the cancellation itself.

`COULD-1`, `CANCEL-1a` to `CANCEL-7`. Quote first: `GET …/cancellation-quote?cancellationDate=yyyy-MM-dd` changes nothing. Then `POST …/cancellation`; the body is optional, and `cancellationDate` defaults to today. Each refund goes back to the method of the payment it reverses (`CANCEL-2`). Figure 7 (DJ6) shows how a term's outcome is decided.

Cancelled on	`ruleApplied`	Refund
Before the start date	`CANCEL-1a`	Full amount
Start date to start + 14 days, inclusive	`CANCEL-1b`	Full amount
Later	`CANCEL-1c`	Amount × unused days ÷ days in the term
The policy has a claim	`COULD-2`	0.00, whatever the date
The term was never paid (an unpaid renewal)	`UNPAID-TERM`	0.00, and it wins over a claim

1. Quote a cancellation on 2026-11-10, nine days after the start.

```
curl -s -i "http://localhost:5080/policies/GUIDE-001/cancellation-quote?
cancellationDate=2026-11-10"
```

You should see: 200: cooling-off, so the full 803.40 back to the Card.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-001","cancellationDate":"2026-11-10","ruleApplied":"CANCEL-
1b","totalRefund":803.40,"terms":[{"termNumber":1,"ruleApplied":"CANCEL-
1b","daysInTerm":365,"daysUsed":10,"unusedDays":355,"refundAmount":803.40}],"ref
unds":[{"type":"Card","amount":803.40,"paymentReference":"PAY-
3e6b82d3846e4b3784ea7086a0066122"}],"quotedOn":"2026-10-25"}
```

2. Quote 2027-04-30, after cooling-off.

```
curl -s -i "http://localhost:5080/policies/GUIDE-001/cancellation-quote?
cancellationDate=2027-04-30"
```

You should see: 200: pro rata. 181 days used, 184 unused, 803.40 × 184 ÷ 365 = 405.00.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-001","cancellationDate":"2027-04-30","ruleApplied":"CANCEL-
1c","totalRefund":405.00,"terms":[{"termNumber":1,"ruleApplied":"CANCEL-
1c","daysInTerm":365,"daysUsed":181,"unusedDays":184,"refundAmount":405.00}],"re
funds":[{"type":"Card","amount":405.00,"paymentReference":"PAY-
3e6b82d3846e4b3784ea7086a0066122"}],"quotedOn":"2026-10-25"}
```

3. Cancel for 2026-11-10.

```
curl -s -i -X POST http://localhost:5080/policies/GUIDE-001/cancellation -H
"Content-Type: application/json" -d '{"cancellationDate":"2026-11-10"}'
```

You should see: 201, no `Location`. The answer agrees with the quote and now carries a refund `reference`.

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-001","cancellationDate":"2026-11-10","ruleApplied":"CANCEL-
1b","totalRefund":803.40,"terms":[{"termNumber":1,"ruleApplied":"CANCEL-
1b","daysInTerm":365,"daysUsed":10,"unusedDays":355,"refundAmount":803.40}],"ref
unds":[{"reference":"REF-
f85cc5de8f26490fa92129ad1fa4e8c5","type":"Card","amount":803.40,"paymentReferenc
e":"PAY-3e6b82d3846e4b3784ea7086a0066122"}]}
```

4. Cancel again, then cancel the other policy for yesterday.

```
curl -s -i -X POST http://localhost:5080/policies/GUIDE-001/cancellation
curl -s -i -X POST http://localhost:5080/policies/GUIDE-002/cancellation -H
"Content-Type: application/json" -d '{"cancellationDate":"2026-10-24"}'
```

You should see: The first is 409 **`CANCEL-4`**, which gives the original cancellation date. The second is 422 **`CANCEL-3`**: no past dates.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/cancel-4","title":"Policy already
cancelled","status":409,"detail":"The policy was already cancelled on 2026-11-
10.","instance":"/policies/GUIDE-001/cancellation","traceId":"00-
834ec333bb3c73ca16050e9cfab120e9-fece7dd538b051d0-00","ruleId":"CANCEL-
4","cancellationDate":"2026-11-10"}
```

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/cancel-3","title":"Cancellation date in the
past","status":422,"detail":"The cancellation date cannot be in the
past.","instance":"/policies/GUIDE-002/cancellation","traceId":"00-
a01771ccb715cac17879cc146f9658d3-e89072f53e919917-00","ruleId":"CANCEL-
3","today":"2026-10-25","violations":[{"ruleId":"CANCEL-3","detail":"The
cancellation date cannot be in the past."}]}
```

Failure	Status	Rule
Policy already cancelled	409	CANCEL-4
Date before today; date after the end of cover	422	CANCEL-3 CANCEL-7
Unknown reference	404	GET-1
Quote without `cancellationDate`, a bad date format, an unknown query key, or an unknown JSON field	400	none (shape)

In Postman: folder **F3 Cancel**, requests `COULD-1 · quote then cancel for the same date agree → 201` and `CANCEL-4 · cancel twice → 409`.

You should see: The quote and the cancellation for 2026-11-10 agree: `ruleApplied` `CANCEL-1b` and `totalRefund` 803.40.

Claim, renew, read errors and run it all (SJ5 to SJ8)

SJ5 · Record a claim

> **You decide** The date of the claim and a description.

> **The API gives you** The recorded claim, and a changed refund: after any claim every later quote is £0.

COULD-2, COULD-2a to COULD-2c. `POST …/claims` takes a required `claimDate` and an optional `description` of up to 500 characters. The date must not be in the future and must fall inside a term's cover. The claim goes on GUIDE-002, sold in SJ2.

1. Quote a cancellation for 2027-04-30 on GUIDE-002 before any claim.

```
curl -s -i "http://localhost:5080/policies/GUIDE-002/cancellation-quote?
cancellationDate=2027-04-30"
```

You should see: 200: pro rata, 188 days used and 177 unused. 1,200.00 × 177 ÷ 365 = 581.92.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-002","cancellationDate":"2027-04-30","ruleApplied":"CANCEL-
1c","totalRefund":581.92,"terms":[{"termNumber":1,"ruleApplied":"CANCEL-
1c","daysInTerm":365,"daysUsed":188,"unusedDays":177,"refundAmount":581.92}],"re
funds":[{"type":"DirectDebit","amount":581.92,"paymentReference":"PAY-
0bcdeff62189401c98fed43d783c842e"}],"quotedOn":"2026-10-25"}
```

2. Record a claim for today.

```
curl -s -i -X POST http://localhost:5080/policies/GUIDE-002/claims -H "Content-
Type: application/json" -d '{"claimDate":"2026-10-25","description":"Burst
pipe"}'
```

You should see: 201 with `Location` pointing at the policy, not at a claim.

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /policies/GUIDE-002

{"reference":"CLM-GUIDE-002-1","policyReference":"GUIDE-002","claimDate":"2026-
10-25","description":"Burst pipe","hasClaims":true}
```

3. Quote the same date again.

```
curl -s -i "http://localhost:5080/policies/GUIDE-002/cancellation-quote?
cancellationDate=2027-04-30"
```

You should see: 200 and nothing back: **COULD-2**. No `refunds` are listed, because no £0 refund record is ever created.

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{"reference":"GUIDE-002","cancellationDate":"2027-04-30","ruleApplied":"COULD-
2","totalRefund":0.00,"terms":[{"termNumber":1,"ruleApplied":"COULD-
2","daysInTerm":365,"daysUsed":188,"unusedDays":177,"refundAmount":0.00}],"refun
ds":[],"quotedOn":"2026-10-25"}
```

4. Try a claim for tomorrow, then a claim on the policy that was cancelled in SJ4.

```
curl -s -i -X POST http://localhost:5080/policies/GUIDE-002/claims -H "Content-
Type: application/json" -d '{"claimDate":"2026-10-26"}'
curl -s -i -X POST http://localhost:5080/policies/GUIDE-001/claims -H "Content-
Type: application/json" -d '{"claimDate":"2026-11-10"}'
```

You should see: The first is 422 `COULD-2a`. The second is 409 `COULD-2c`.

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/could-2a","title":"Claim date in the
future","status":422,"detail":"The claim date cannot be in the
future.","instance":"/policies/GUIDE-002/claims","traceId":"00-
4fedf139ee97b7cda5a2a3956fa771af-fb8571469aaaa359-00","ruleId":"COULD-
2a","today":"2026-10-25","violations":[{"ruleId":"COULD-2a","detail":"The claim
date cannot be in the future."}]}
```

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json; charset=utf-8

{"type":"/problems/could-2c","title":"Claim on a cancelled
policy","status":409,"detail":"The policy was cancelled on 2026-11-10, so a
claim cannot be recorded.","instance":"/policies/GUIDE-
001/claims","traceId":"00-4e27ceb1f25124890d8c893d7ce77f63-167e582836c17253-
00","ruleId":"COULD-2c","cancellationDate":"2026-11-10"}
```

> **A claim cannot be undone**
> It sets `hasClaims` for good, and every later refund is £0. A claim is not idempotent: sending it twice records two. That is why the guide shows it only on a throw-away policy.

Failure	Status	Rule
Claim date after today; outside every term's cover	422	`COULD-2a` `COULD-2b`
Policy is cancelled	409	`COULD-2c`
No `claimDate`; description over 500 characters	400	none (shape)

In Postman: folder **F3 Cancel**, request `COULD-2 · claim then cancel refunds nothing → 201`.

You should see: The same quote date gave 581.92 before the claim and 0.00 after it: `ruleApplied` changed from `CANCEL-1c` to `COULD-2`.

SJ6 · Renew

> **You decide** Whether to renew inside the window, and the new term's amount and payment method.

> **The API gives you** The policy with a second term, or a refusal that names the window (RENEW-1/2) or the broken rule.

`RENEW-1/2` to `RENEW-6`, `COULD-3`. `POST …/renewals` takes the new term's `amount` and, optionally, a `paymentMethod`. You may renew from 30 days before the latest term's end date to the end date, both inclusive. For GUIDE-002, which ends 2027-10-24, that is 2027-09-24 to 2027-10-24. The new term starts the day after the end. With `autoRenew` on, the payment method defaults to the current term's, and a Cheque is refused. With it off, the term is created unpaid and a `paymentMethod` is refused.

1. Renew the policy sold today (GUIDE-002).

   ```
   curl -s -i -X POST http://localhost:5080/policies/GUIDE-002/renewals -H
   "Content-Type: application/json" -d '{"amount":1250.00}'
   ```

   You should see: 422: `RENEW-1/2`. The window is named in the body.

   ```
   HTTP/1.1 422 Unprocessable Entity
   Content-Type: application/problem+json; charset=utf-8

   {"type":"/problems/renew-1-2","title":"Outside the renewal
   window","status":422,"detail":"A policy can be renewed from 2027-09-24 to 2027-
   10-24.","instance":"/policies/GUIDE-002/renewals","traceId":"00-
   73c6ea53cc4839eb91533522a86cf489-a3ab46fc4cd1126d-00","ruleId":"RENEW-
   1/2","windowStart":"2027-09-24","windowEnd":"2027-10-24","violations":
   [{"ruleId":"RENEW-1/2","detail":"A policy can be renewed from 2027-09-24 to
   2027-10-24."}]}
   ```

> **A successful renewal is not shown live**
> A policy can only be sold from today, so it cannot be inside its renewal window for 335 days, and the API runs on the real clock. The acceptance tests set the clock instead: the 13 tests traited `RENEW-4` passed in this run (`dotnet test --solution Uinsure.Policies.slnx -- --filter-trait "Rule=RENEW-4"`). A renewal returns the policy with a second entry in `terms` . Postman's **F4 Renew** has no happy-path requests for the same reason.

Failure	Status	Rule
Outside the window; amount 0 or below	422	`RENEW-1/2` `RENEW-5`
`paymentMethod` sent with auto-renew off; a Cheque with auto-renew on	422	`RENEW-3b` `COULD-3`
Policy is cancelled	409	`RENEW-6`
No body, no amount, a start date sent (the client does not choose it), a bad payment method	400	none (shape)

In Postman: folder **F4 Renew**, request `RENEW-3b · auto-renew off with a paymentMethod, today → 422` .

You should see: The refusal body above: `windowStart` 2027-09-24 and `windowEnd` 2027-10-24 are the dates you may renew between.

SJ7 · Read an error

> **You decide** Whether to fix the input and retry, or to stop.

> **The API gives you** One RFC 9457 body for every failure, a status that says what kind of problem it is, and every broken 422 rule at once.

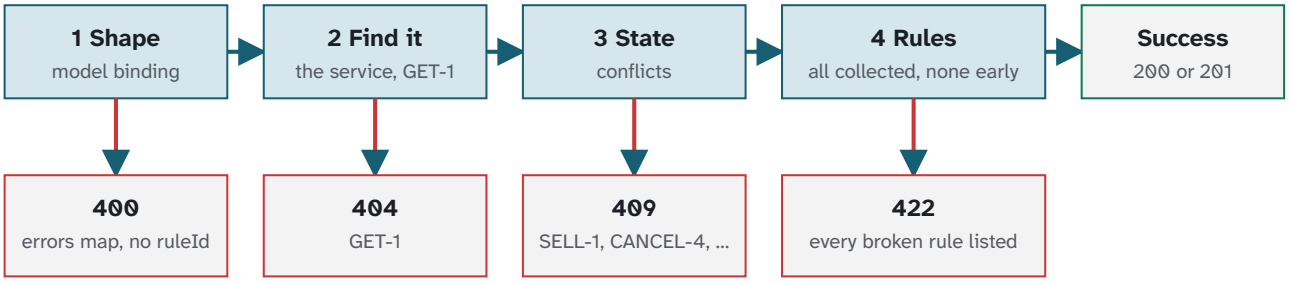
Every body follows RFC 9457. `type` is `/problems/` and the rule ID in lower case, with `/` as `-`. `ruleId` names the rule, `violations[]` lists every broken rule on a 422, and `traceId` finds the request in the logs. One place maps a rule to its status, type and title.

`src/Uinsure.Policies.Api/Errors/ProblemCatalog.cs:30`

```
    public static int StatusFor(ViolationKind kind) => kind switch
    {
        ViolationKind.NotFound => StatusCodes.Status404NotFound,
        ViolationKind.Conflict => StatusCodes.Status409Conflict,
        _ => StatusCodes.Status422UnprocessableEntity,
    };

    public static string TypeFor(string ruleId) => "/problems/" +
ruleId.ToLowerInvariant().Replace('/', '-');

    public static string TitleFor(string ruleId) => Titles[ruleId];
```



The first three stop the request alone. Only stage 4 reports more than one failure.

Figure 10. The order a request can fail in, and the status each stage gives.

Status	What kind of problem	Carries
400	The request has the wrong shape: a missing or unknown field, a wrong type, a bad date, an unknown query key	`errors` map, `traceId`, no `ruleId`
404	Unknown reference (`GET-1`)	`ruleId`
409	State conflict: `SELL-1`, `CANCEL-4`, `RENEW-6`, `COULD-2c`. Also two writes on one policy (`/problems/concurrency`, no rule)	`ruleId` (except concurrency)
422	A business rule is broken	`ruleId` and `violations[]`
429	Rate limit per client IP; `/problems/rate-limited`	`Retry-After` header
500	Anything unexpected	Generic `detail` and `traceId`. The exception goes to the logs only

1. See the order at work: sell a policy whose reference is taken *and* whose start date is in the past, using a lower-case reference.

   ```
   curl -s -i -X POST http://localhost:5080/policies -H "Content-Type:
   application/json" -d '{"reference":"guide-
   001","type":"Household","startDate":"2026-10-01",…}'
   ```

   You should see: 409 `SELL-1` alone. The past start date is not mentioned, because a 409 is returned before any 422 rule is looked at:

   ```
   HTTP/1.1 409 Conflict
   Content-Type: application/problem+json; charset=utf-8

   {"type":"/problems/sell-1","title":"Policy reference already
   exists","status":409,"detail":"A policy with reference 'GUIDE-001' already
   exists.","instance":"/policies","traceId":"00-4d0b5ba6bc8277c1394cc0a338b74ec1-
   568966fb0d263905-00","ruleId":"SELL-1","reference":"GUIDE-001"}
   ```

> **Evaluation order**
> 400 shape, then 404, then 409, each returned alone. Only then are all the 422 rules checked together and listed. So a duplicate reference with a bad start date is a 409, and a bad start date with a bad amount is one 422 with two violations (the response in SJ2). Remove the duplicate and the same body gives the 422.

A 429, a 500, and a 413 for a body over 65,536 bytes are covered by the acceptance tests and were not provoked live for this guide.

You should see: The 409 above names only SELL-1, and the two-violation 422 in SJ2 names SELL-2 and SELL-7: Figure 10 in action.

SJ8 · Run the whole collection

> **You decide** Whether to run it in the Postman app or the CLI, and the whole collection or one folder.

> **The API gives you** A table of requests, scripts and assertions, with how many failed and how long the run took.

Every request stands alone: its pre-request script makes a fresh reference and today's dates and sells whatever it needs, so order does not matter and nothing needs resetting. Each test name starts with its rule ID.

The Postman CLI (MAN-2)

1. Start the API (DJ2, step 3). Then, from the repository root, run one folder with `-i`, or leave `-i` out to run the whole collection. No login is needed.

   ```
   postman collection run tests/postman/Uinsure.Policies.postman_collection.json -e
   tests/postman/local.postman_environment.json -i "F2 Retrieve"
   ```

   You should see: A summary table. The counts are requests, scripts and assertions executed and failed; a failure would appear in the `failed` column:

   ```
   ┌─────────────────────────┬────────────────────┬────────────────────┐
   │                         │           executed │             failed │
   ├─────────────────────────┼────────────────────┼────────────────────┤
   │              iterations │                  1 │                  0 │
   ├─────────────────────────┼────────────────────┼────────────────────┤
   │                requests │                 25 │                  0 │
   ├─────────────────────────┼────────────────────┼────────────────────┤
   │            test-scripts │                 14 │                  0 │
   ├─────────────────────────┼────────────────────┼────────────────────┤
   │      prerequest-scripts │                 19 │                  0 │
   ├─────────────────────────┼────────────────────┼────────────────────┤
   │              assertions │                 26 │                  0 │
   ├─────────────────────────┴────────────────────┴────────────────────┤
   │ total run duration: 1867 ms                                       │
   └───────────────────────────────────────────────────────────────────┘
   ```

> **Provenance**
> This output was captured on the F8 run of 2026-10-23, against the same collection, with the base URL overridden to the API port in use then. The Postman CLI could not start on the machine that reprinted this guide: it stopped with a Private Cloud manifest error from the machine's registry policy. So no whole-collection count is shown here, and the Postman app was not run either.

> **Reading the numbers**
> The F2 Retrieve folder holds 14 requests, yet the run executed 25: the set-up sales and read-backs that each request makes are requests too. *Test scripts* (14) are the requests that assert something, and *assertions* (26) are the checks inside them.

The Postman app (MAN-1)

1. On the collection, choose ··· → **Run collection**, then **Run**. The Collection Runner lists each request with its passed assertions.
2. Run a folder with ··· → **Run folder**, or one of its `happy path` or `rule violations` sub-folders.

3. Run one request: open it and choose **Send**, then read the **Test Results** tab.

PLACEHOLDER: screenshot to be taken by hand

Postman: Collection Runner results (figure 11)

Figure 11. The Collection Runner after a full run. Placeholder: a hand capture.

Folder	Requests	Covers	Rule IDs
F0b Walking skeleton	2	Health, live and ready	none
F1 Sell	39	Happy path and violations of the sale	`SELL-1` to `SELL-7`, shape checks
F1b Layered models	5	Amount scale and limits, round trips through storage	shape checks
F2 Retrieve	14	Read-back, status, 404	`GET-1`, shape checks
F3 Cancel	33	Quote, cancel, refunds, claims	`CANCEL-1a` to `CANCEL-7`, `COULD-1`, `COULD-2` to `COULD-2c`
F4 Renew	18	Renewal failures on the real clock	`RENEW-1/2` to `RENEW-6`, `COULD-3`

You should see: The summary table: nothing in the `failed` column, and the duration beside it.

Check your understanding

Question	Answer
SQ1. How do I try it without writing code?	The Postman app or CLI collection, or the Scalar page at `/scalar/v1` in Development.
SQ2. How do I sell a policy?	`POST /policies` with a reference, start date, amount, payment method, policyholders and property.
SQ3. How do I look one up, and what does its status mean?	`GET /policies/{reference}`. Status is derived from today: Pending, Active, Expired, or Cancelled.
SQ4. How do I cancel, and how much comes back?	Quote with `GET …/cancellation-quote`, then `POST …/cancellation`. Full refund to start + 14, pro rata after, 0 with a claim.
SQ5. What does a claim do?	It sets `hasClaims`, and every later refund is £0.
SQ6. When can I renew?	From 30 days before the end date to the end date.

Question	Answer
SQ7/SQ8. How do I read an error, and what does the status say?	Read `ruleId` and `violations[]`. 400 shape, 404 unknown, 409 conflict, 422 broken rule, 429 slow down, 500 not you.
SQ9. Why was my duplicate or backdated request refused?	A duplicate reference (any case) is 409 **SELL-1**. A start date before today or after today + 60 is 422 **SELL-2**.
SQ10. Which rule decided my refund?	`ruleApplied` on each term: CANCEL-1a, CANCEL-1b, CANCEL-1c, COULD-2 or UNPAID-TERM.
SQ11. Why do I get several violations at once?	After the 400, 404 and 409 checks (each returned alone), every 422 rule is collected.
SQ12. How do I run the whole collection?	`postman collection run` with the local environment, then read the requests and assertions in the summary.
SQ14. What does the API give me that I would otherwise build by hand?	Server-side dates and status, refunds per term with the rule that chose them, one error shape, and rate-limit and size guards.

Troubleshooting

Symptoms met while building and capturing this guide, and what to do.

1 TJ1 · Find the symptom, apply the fix

> **You decide** Whether the fix is yours to make (a port, a path, a request) or a finding to report.

> **The guide gives you** The cause of each problem met while the guide was captured, and the fix.

Symptom	Cause	Fix
"The filename or extension is too long" in the architecture tests	The clone path is long, and SQLite's native library is copied deep under each test project's `bin`	Clone to a short path such as `C:\src\uinsure-policies`
"address already in use" on `dotnet run`	Visual Studio or another run holds port 5080	Stop the other run
`dotnet run` fails with "Database:Provider must be Sqlite or SqlServer"	It ran with no launch profile, so it was not Development and had no database configuration (met while capturing this guide)	Use `--launch-profile http`
Postman: connection refused	The API is not running, or `baseUrl` is not `http://localhost:5080`	Start the API; select the **local** environment

Symptom	Cause	Fix
The Postman CLI stops with a Private Cloud manifest error	A registry policy on the machine names a manifest the CLI cannot read	Ask whoever manages the machine, or use the Postman app
400 on `GET /policies/{reference}?reference=X`	A query key named like a route parameter is unknown, and unknown keys are refused	Remove the query key
400 `/problems/https-required`	Plain HTTP outside Development	Call the HTTPS address
429 with `Retry-After`	The per-client rate limit	Wait the number of seconds given
422 on a sale that worked yesterday	The start date is now in the past (`SELL-2`)	Send a start date from today to today + 60
409 `/problems/concurrency`	Two writes on one policy at once	Retry the request
`curl` prints a web page or an error in PowerShell	In PowerShell, `curl` is an alias for another command, and quoting differs	Use Git Bash, or call `curl.exe`
The API and web profile stops with "Packages are not restored"	A fresh clone has no `node_modules` for Docket	`npm --prefix src/Uinsure.Policies.Web ci`, then start again. Docket needs Node 22.22 or later
`dotnet ef` warns "tools version '9.0.9' is older than that of the runtime '10.0.13'"	An older global `dotnet-ef`; the repository has no tool manifest	Update the global `dotnet-ef` to the runtime's version. The script is still generated
A warning in the log about "QuerySplittingBehavior" on the first read	EF Core notes a query that loads several collections. It is a performance note, not a failure	None needed

You should see: A fix for each row, and the rule or status it points to where one applies.

Check your understanding

Question	Answer
SQ13. What goes wrong when I start, and how do I fix it?	Port 5080 in use, connection refused, an HTTPS rejection or a rate limit: the table above gives the cause and the fix for each.
SQ8. Why did I get a 429 or a 500?	429: too many requests from one address, wait for `Retry-After`. 500: an unexpected server error; quote the `traceId`.